런타임 로더 (Runtime Loader)

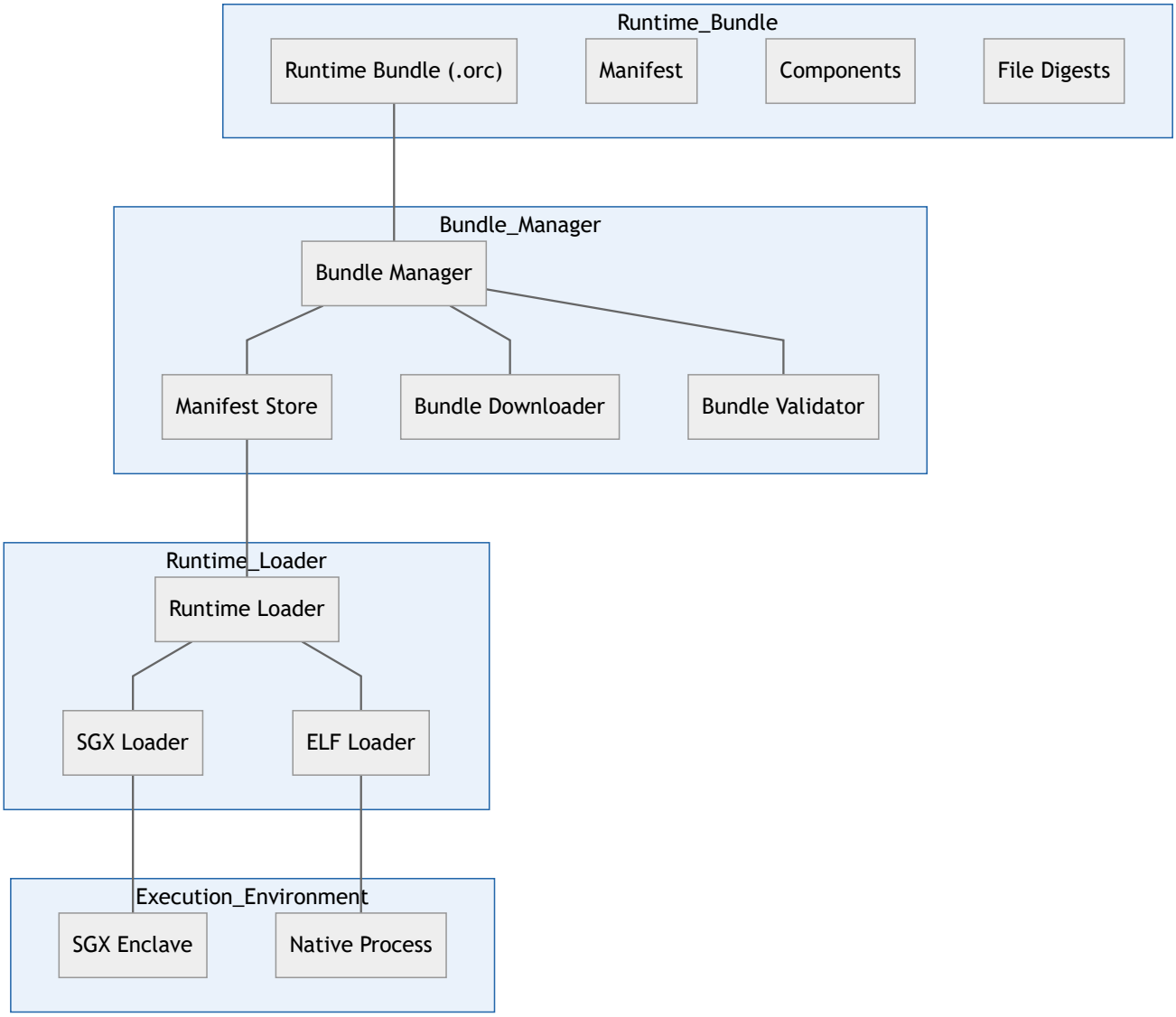
소개

런타임 로더는 Oasis Core에서 런타임 바이너리를 로딩하고 실행하는 핵심 컴포넌트입니다. 런타임 로더는 런타임 번들에서 적절한 실행 파일을 추출하고, 필요한 검증을 수행한 후, 지정된 실행 환경에서 런타임을 시작합니다.

런타임 로더는 다양한 런타임 형태(SGX 엔클레이브, ELF 바이너리)를 지원하며, 런타임 번들의 무결성을 보장하고 버전 관리를 담당합니다.

아키텍처

전체 구조



주요 컴포넌트

1. **Runtime Bundle**: 런타임 실행 파일과 메타데이터를 포함하는 패키지
2. **Bundle Manager**: 런타임 번들의 다운로드, 검증, 관리를 담당
3. **Runtime Loader**: 실제 런타임 바이너리 로딩과 실행을 담당
4. **Manifest Store**: 런타임 매니페스트 정보를 저장하고 관리

런타임 번들 구조

번들 파일 형식

런타임 번들은 `.orc` (Oasis Runtime Container) 확장자를 가진 ZIP 아카이브 파일입니다.

번들 내부 구조

```
runtime.orc
├── META-INF/
| └── MANIFEST.MF # 매니페스트 파일 (JSON)
├── runtime.elf # ELF 실행 파일
├── runtime.sgxs # SGX 엔클레이브 파일
├── runtime.sig # SGX 서명 파일
└── other-assets/ # 기타 자산 파일들
```

매니페스트 구조

기본 매니페스트

```go
type Manifest struct {
// 런타임 이름 (선택사항)
Name  string  `json:"name,omitempty"`
// 런타임 식별자
ID common.Namespace `json:"id"`
// 런타임 버전 (레거시)
```

```
Version version.Version `json:"version,omitempty"`
// 실행 파일 이름 (레거시)
Executable  string  `json:"executable,omitempty"`
// SGX 메타데이터 (레거시)
SGX *SGXMetadata `json:"sgx,omitempty"`
// 런타임 컴포넌트들
Components []*Component `json:"components,omitempty"`
// 파일 다이제스트 맵
Digests  map[string]hash.Hash `json:"digests"`
}
```

컴포넌트 구조

```
type Component struct {
// 컴포넌트 종류 (RONL, ROFL)
Kind component.Kind `json:"kind"`
// 컴포넌트 이름
Name  string  `json:"name,omitempty"`
// 컴포넌트 버전
Version version.Version `json:"version,omitempty"`
// ELF 메타데이터
ELF *ELFMetadata `json:"elf,omitempty"`
// SGX 메타데이터
SGX *SGXMetadata `json:"sgx,omitempty"`
```

```
// TDX 메타데이터
TDX *TDXMetadata `json:"tdx,omitempty"`
// 예상 엔클레이브 신원
Identities []Identity `json:"identity,omitempty"`
// 기본적으로 비활성화 여부
Disabled bool `json:"disabled,omitempty"`
}
```

ELF 메타데이터

```
type ELFMetadata struct {
// ELF 실행 파일 이름
Executable string `json:"executable"`
}
```

SGX 메타데이터

```
type SGXMetadata struct {
// SGX 엔클레이브 실행 파일 이름
Executable string `json:"executable"`
// SGX 엔클레이브 서명 파일 이름
Signature string `json:"signature"`
}
```

번들 검증

1. 매니페스트 검증

```
func (bnd *Bundle) Validate() error {
// 매니페스트 유효성 검사
if err := bnd.Manifest.Validate(); err != nil {
return fmt.Errorf("malformed manifest: %w", err)
}
// 필수 파일 존재 확인
for id, comp := range bnd.Manifest.GetAvailableComponents() {
if elf := comp.ELF; elf != nil {
if _, ok := bnd.Data[elf.Executable]; !ok {
return fmt.Errorf("missing ELF executable for %s", id)
}
}
if sgx := comp.SGX; sgx != nil {
if _, ok := bnd.Data[sgx.Executable]; !ok {
return fmt.Errorf("missing SGX executable for %s", id)
}
}
}
// 파일 다이제스트 검증
for fn, d := range bnd.Data {
```

```go
h, err := HashAllData(d)
if err != nil {
return fmt.Errorf("failed to read '%s': %w", fn, err)
}
mh, ok := bnd.Manifest.Digests[fn]
if !ok && fn != manifestName {
return fmt.Errorf("missing digest: '%s'", fn)
}
if ok && !h.Equal(&mh) {
return fmt.Errorf("invalid digest: '%s'", fn)
}
}
return nil
}
```

2. SGX 서명 검증

```go
func (bnd *Bundle) verifySgxSignature(comp *Component) error {
if comp.SGX == nil || comp.SGX.Signature == "" {
return nil
}
// MRENCLAVE 계산
mrEnclave, err := bnd.MrEnclave(comp.ID())
if err != nil {
```

```go
return err
}
// 서명 파일 읽기
d, ok := bnd.Data[comp.SGX.Signature]
if !ok {
return fmt.Errorf("no SGX signature for '%s'", comp.ID())
}
b, err := ReadAllData(d)
if err != nil {
return fmt.Errorf("failed to read SGX signature: %w", err)
}
// SIGSTRUCT 검증
_, sigStruct, err := sigstruct.Verify(b)
if err != nil {
return fmt.Errorf("failed to verify sigstruct: %w", err)
}
// 엔클레이브 해시 일치 확인
if sigStruct.EnclaveHash != *mrEnclave {
return fmt.Errorf("sigstruct does not match SGXS")
}
return nil
}
```

번들 관리

번들 매니저

매니저 초기화

```
func NewManager(dataDir string, runtimeIDs []common.Namespace, store ManifestS
// HTTP 클라이언트 설정
client := http.Client{
Timeout: requestTimeout,
}
// 번들 크기 제한 설정
bundleSize := int64(maxDefaultBundleSizeBytes)
if size := config.GlobalConfig.Runtime.MaxBundleSize; size != "" {
bundleSize = int64(config.ParseSizeInBytes(size))
}
// 레지스트리 URL 검증
globalBaseURLs, err := validateAndNormalizeURLs(config.GlobalConfig.Runtime.Regi
if err != nil {
return nil, err
}
return &Manager{
dataDir: dataDir,
bundleDir: ExplodedPath(dataDir),
```

```
tmpBundleDir: TmpBundlePath(dataDir),
maxBundleSizeBytes: bundleSize,
runtimeIDs: runtimes,
globalBaseURLs: globalBaseURLs,
runtimeBaseURLs: runtimeBaseURLs,
client: &client,
store: store,
volumeManager: volumeManager,
}, nil
}
```

번들 다운로드

다운로드 프로세스

번들 매니저는 다음 단계를 통해 런타임 번들을 다운로드합니다:

1. **매니페스트 해시 확인**: 이미 다운로드된 번들인지 확인
2. **레지스트리 순회**: 런타임별 레지스트리와 글로벌 레지스트리에서 순차적으로 시도
3. **메타데이터 다운로드**: 번들의 실제 다운로드 URL 획득
4. **번들 다운로드**: 실제 번들 파일 다운로드
5. **검증 및 압축 해제**: 번들 무결성 검증 후 디스크에 압축 해제
6. **매니페스트 등록**: 매니페스트 스토어에 등록

```
func (m *Manager) downloadBundle(runtimeID common.Namespace, manifestHash hash.H
// 이미 다운로드된 번들인지 확인
```

```
if m.store.HasManifest(manifestHash) {
m.logger.Debug("번들이 이미 존재함", "manifest_hash", manifestHash.Hex())
return nil
}
m.logger.Info("번들 다운로드 시작",
"runtime_id", runtimeID,
"manifest_hash", manifestHash.Hex(),
)
// 모든 레지스트리 URL에서 시도
registryLists := [][]string{m.runtimeBaseURLs[runtimeID], m.globalBaseURLs}
for i, baseURLs := range registryLists {
registryType := "runtime-specific"
if i == 1 {
registryType = "global"
}
for _, baseURL := range baseURLs {
m.logger.Debug("레지스트리에서 번들 다운로드 시도",
"registry_type", registryType,
"base_url", baseURL,
)
if err := m.tryDownloadBundle(manifestHash, baseURL); err != nil {
m.logger.Warn("번들 다운로드 실패",
"base_url", baseURL,
```

```go
"err", err,
)
continue
}
m.logger.Info("번들 다운로드 성공", "base_url", baseURL)
return nil
}
}
return fmt.Errorf("모든 레지스트리에서 번들 다운로드 실패")
}

func (m *Manager) tryDownloadBundle(manifestHash hash.Hash, baseURL string) err
// 메타데이터 URL 구성 (manifest hash를 이용)
metaURL, err := url.JoinPath(baseURL, manifestHash.Hex())
if err != nil {
return fmt.Errorf("메타데이터 URL 구성 실패: %w", err)
}
// 메타데이터 다운로드 (실제 번들 URL 획득)
bundleURL, err := m.fetchMetadata(metaURL)
if err != nil {
return fmt.Errorf("메타데이터 다운로드 실패: %w", err)
}
// 번들 파일 다운로드
```

```go
src, err := m.fetchBundle(bundleURL)
if err != nil {
return fmt.Errorf("번들 다운로드 실패: %w", err)
}
defer os.Remove(src) // 임시 파일 정리
// 번들 압축 해제 및 검증
manifest, err := m.explodeBundle(src, WithBundleManifestHash(manifestHash))
if err != nil {
return fmt.Errorf("번들 압축 해제 실패: %w", err)
}
// 매니페스트 스토어에 등록
return m.registerManifest(manifest)
}


// 메타데이터 다운로드 (번들 URL 획득)
func (m *Manager) fetchMetadata(metaURL string) (string, error) {
resp, err := m.client.Get(metaURL)
if err != nil {
return "", fmt.Errorf("HTTP 요청 실패: %w", err)
}
defer resp.Body.Close()
if resp.StatusCode != http.StatusOK {
return "", fmt.Errorf("HTTP 오류: %d", resp.StatusCode)
```

```go
}
// 메타데이터 크기 제한 확인
if resp.ContentLength > maxMetadataSizeBytes {
return "", fmt.Errorf("메타데이터 크기 초과: %d bytes", resp.ContentLength)
}
// 메타데이터 읽기
body, err := io.ReadAll(io.LimitReader(resp.Body, maxMetadataSizeBytes))
if err != nil {
return "", fmt.Errorf("메타데이터 읽기 실패: %w", err)
}
// JSON 파싱하여 번들 URL 추출
var metadata struct {
BundleURL string `json:"bundle_url"`
}
if err := json.Unmarshal(body, &metadata); err != nil {
return "", fmt.Errorf("메타데이터 파싱 실패: %w", err)
}
return metadata.BundleURL, nil
}


// 번들 파일 다운로드
func (m *Manager) fetchBundle(bundleURL string) (string, error) {
resp, err := m.client.Get(bundleURL)
```

```go
if err != nil {
return "", fmt.Errorf("번들 다운로드 HTTP 요청 실패: %w", err)
}
defer resp.Body.Close()
if resp.StatusCode != http.StatusOK {
return "", fmt.Errorf("번들 다운로드 HTTP 오류: %d", resp.StatusCode)
}
// 번들 크기 제한 확인
if resp.ContentLength > m.maxBundleSizeBytes {
return "", fmt.Errorf("번들 크기 초과: %d bytes", resp.ContentLength)
}
// 임시 파일 생성
tmpFile, err := os.CreateTemp(m.tmpBundleDir, "bundle-*.orc")
if err != nil {
return "", fmt.Errorf("임시 파일 생성 실패: %w", err)
}
defer tmpFile.Close()
// 번들 데이터를 임시 파일에 저장
_, err = io.Copy(tmpFile, io.LimitReader(resp.Body, m.maxBundleSizeBytes))
if err != nil {
os.Remove(tmpFile.Name())
return "", fmt.Errorf("번들 저장 실패: %w", err)
}
```

```
return tmpFile.Name(), nil
}
```

번들 압축 해제

번들 압축 해제 과정

```
func (m *Manager) explodeBundle(path string, opts ...AddOption) (*ExplodedManif
options := NewAddOptions(opts...)
// 번들 열기
var openOpts []OpenOption
if options.manifestHash != nil {
openOpts = append(openOpts, WithManifestHash(*options.manifestHash))
}
bnd, err := Open(path, openOpts...)
if err != nil {
return nil, fmt.Errorf("failed to open bundle: %w", err)
}
defer bnd.Close()
// 번들 검증
if options.validator != nil {
if err = options.validator(bnd); err != nil {
return nil, err
}
```

```
}

// 매니페스트 재작성 (필요시)

bnd.Rewrite(options.manifestRewriter)

// 디스크에 압축 해제

dir := bnd.ExplodedPath(m.dataDir)

if err = bnd.WriteExploded(dir); err != nil {

return nil, fmt.Errorf("failed to explode bundle: %w", err)

}

return &ExplodedManifest{

Manifest: bnd.Manifest,

ExplodedDataDir: dir,

Labels: options.labels,

Volumes: options.volumes,

}, nil

}
```

압축 해제된 번들 구조

```
/data/bundles/

└── <manifest-hash>/

├── META-INF/

| └── MANIFEST.MF

├── runtime.elf
```

```
├── runtime.sgxs
├── runtime.sig
└── other-assets/
```

런타임 로더 구현

로더 인터페이스

기본 로더 트레이트

```
/// 런타임 로더 트레이트
/// 다양한 런타임 형태(SGX, ELF 등)를 지원하는 통합 인터페이스
pub trait Loader {
/// 지정된 런타임을 로드하고 실행
///
/// # 매개변수
/// - `filename`: 런타임 실행 파일 경로
/// - `signature_filename`: SGX 서명 파일 경로 (SGX 런타임의 경우 필수)
/// - `host_socket`: 워커 호스트와 통신할 Unix 소켓 경로
/// - `allow_network`: 네트워크 접근 허용 여부
fn run(
&self,
filename: &str,
signature_filename: Option<&str>,
```

```
host_socket: &str,
allow_network: bool,
) -> Result<()>;
}
```

로더 팩토리

```
/// 런타임 타입에 따른 적절한 로더 생성
pub fn create_loader(runtime_type: &str) -> Box<dyn Loader> {
match runtime_type {
#[cfg(target_os = "linux")]
"sgxs" => Box::new(SgxsLoader),
#[cfg(feature = "elf")]
"elf" => Box::new(ElfLoader),
_ => panic!("지원하지 않는 런타임 타입: {}", runtime_type),
}
}
```

SGX 로더

SGX 로더 구현

```
pub struct SgxsLoader;
```

```
impl Loader for SgxsLoader {
fn run(
&self,
filename: &str,
signature_filename: Option<&str>,
host_socket: &str,
allow_network: bool,
) -> Result<()> {
let sig = signature_filename
.ok_or_else(|| anyhow!("signature file is required"))?;
// SGX 디바이스 초기화
let mut device = IsgxDevice::new()?
.einittoken_provider(AesmClient::new())
.build();
// 엔클레이브 빌더 설정
let mut enclave_builder = EnclaveBuilder::new(filename.as_ref());
enclave_builder.signature(sig)?;
enclave_builder.usercall_extension(HostService::new(host_socket, allow_network))
// 엔클레이브 빌드 및 실행
let enclave = enclave_builder
.build(&mut device)
.map_err(|err| anyhow!("{}", err))?;
enclave.run().map_err(|err| anyhow!("{}", err))
```

```
}

}
```

호스트 서비스 확장

```
struct HostService {

host_socket: String,

allow_network: bool,

}


impl UsercallExtension for HostService {

fn  connect_stream<'future>(

&'future self,

addr: &'future str,

_local_addr: Option<&'future mut String>,

_peer_addr: Option<&'future mut String>,

) -> Pin<Box<dyn Future<Output = IoResult<Option<Box<dyn AsyncStream>>>> + 'futu

async  move {

match  addr {

"worker-host" => {

// 워커 호스트 소켓에 연결

let  stream = UnixStream::connect(self.host_socket.clone()).await?;

let  async_stream: Box<dyn AsyncStream> = Box::new(stream);

Ok(Some(async_stream))
```

```
}
_ if self.allow_network => {
// 네트워크 접근 허용시 기본 핸들러로 전달
Ok(None)
}
_ => {
// 네트워크 접근 불허시 거부
Err(IoError::new(IoErrorKind::Other, "invalid destination"))
}
}
}
.boxed_local()
}
}
```

런타임 로더 실행

메인 로더 프로그램

런타임 로더는 명령행 인터페이스를 통해 실행되며, 다양한 런타임 타입을 지원합니다.

```
use std::path::Path;
use clap::{Arg, ArgAction, Command};
use oasis_core_runtime_loader::Loader;
```

```
fn main() {
// 명령행 인자 정의
let matches = Command::new("Oasis Core Runtime Loader")
.version("1.0.0")
.about("Oasis Core 런타임을 로드하고 실행하는 도구")
.arg(
Arg::new("type")
.long("type")
.help("런타임 타입 (sgxs, elf)")
.value_parser(["sgxs", "elf"])
.default_value("sgxs"),
)
.arg(
Arg::new("runtime")
.value_name("RUNTIME")
.help("런타임 실행 파일 경로")
.required(true)
.index(1),
)
.arg(
Arg::new("signature")
.long("signature")
.help("SGX 서명 파일 경로 (SGX 런타임의 경우 필수)")
```

```
.value_name("FILE"),
)
.arg(
Arg::new("host-socket")
.long("host-socket")
.help("워커 호스트 Unix 소켓 경로")
.value_name("PATH")
.required(true),
)
.arg(
Arg::new("allow-network")
.long("allow-network")
.help("런타임의 네트워크 접근 허용")
.action(ArgAction::SetTrue),
)
.arg(
Arg::new("verbose")
.short('v')
.long("verbose")
.help("상세 로그 출력")
.action(ArgAction::SetTrue),
)
.get_matches();
```

```
// 로깅 초기화
if  matches.get_flag("verbose") {
env_logger::Builder::from_env(env_logger::Env::default().default_filter_or("debu
} else {
env_logger::Builder::from_env(env_logger::Env::default().default_filter_or("info
}
// 인자 파싱 및 검증
let  filename = matches.get_one::<String>("runtime").unwrap();
let  host_socket = matches.get_one::<String>("host-socket").unwrap();
let  mode = matches.get_one::<String>("type").unwrap();
let  signature = matches.get_one::<String>("signature").map(|s| s.as_str());
let  allow_network = matches.get_flag("allow-network");
// 런타임 파일 존재 확인
if !Path::new(filename).exists() {
eprintln!("오류: 런타임 파일을 찾을 수 없습니다: {}", filename);
std::process::exit(1);
}
// SGX 런타임의 경우 서명 파일 필수 확인
if  mode == "sgxs" && signature.is_none() {
eprintln!("오류: SGX 런타임의 경우 --signature 옵션이 필수입니다");
std::process::exit(1);
}
// 서명 파일 존재 확인 (제공된 경우)
```

```
if  let Some(sig_file) = signature {
if !Path::new(sig_file).exists() {
eprintln!("오류: 서명 파일을 찾을 수 없습니다: {}", sig_file);
std::process::exit(1);
}
}
log::info!("런타임 로더 시작");
log::info!("런타임 타입: {}", mode);
log::info!("런타임 파일: {}", filename);
log::info!("호스트 소켓: {}", host_socket);
log::info!("네트워크 허용: {}", allow_network);
if  let Some(sig) = signature {
log::info!("서명 파일: {}", sig);
}
// 적절한 로더 생성
let  loader: Box<dyn Loader> = match  mode.as_str() {
#[cfg(target_os = "linux")]
"sgxs" => {
log::info!("SGX 런타임 로더 초기화");
Box::new(SgxsLoader)
},
#[cfg(not(target_os = "linux"))]
"sgxs" => {
```

```
eprintln!("오류: SGX 로더는 Linux에서만 지원됩니다");
std::process::exit(1);
},
#[cfg(feature = "elf")]
"elf" => {
log::info!("ELF 런타임 로더 초기화");
Box::new(ElfLoader)
},
#[cfg(not(feature = "elf"))]
"elf" => {
eprintln!("오류: ELF 로더가 활성화되지 않았습니다");
std::process::exit(1);
},
_ => {
eprintln!("오류: 지원하지 않는 런타임 타입: {}", mode);
std::process::exit(1);
}
};
// 런타임 실행
log::info!("런타임 실행 시작");
if let Err(e) = loader.run(filename, signature, host_socket, allow_network) {
log::error!("런타임 실행 실패: {}", e);
std::process::exit(1);
```

```
}
log::info!("런타임 실행 완료");
}
```

실행 흐름

1. **명령행 인자 파싱**: 런타임 타입, 파일 경로, 설정 옵션 파싱
2. **파일 존재 확인**: 런타임 파일과 서명 파일 존재 여부 확인
3. **로더 초기화**: 런타임 타입에 따른 적절한 로더 생성
4. **런타임 실행**: 로더를 통한 런타임 시작
5. **오류 처리**: 실행 중 발생하는 오류 처리 및 로깅

런타임 버전 관리

버전 관리 시스템

버전 정보 구조

```
type version.Version struct {
Major  uint64  `json:"major"`
Minor  uint64  `json:"minor"`
Patch  uint64  `json:"patch"`
}
```

버전 비교

```go
func (v Version) Less(other Version) bool {
if v.Major != other.Major {
return v.Major < other.Major
}
if v.Minor != other.Minor {
return v.Minor < other.Minor
}
return v.Patch < other.Patch
}
```

런타임 업그레이드

업그레이드 프로세스

1. **새 버전 다운로드**: 레지스트리에서 새 런타임 번들 다운로드
2. **검증**: 번들 무결성 및 서명 검증
3. **압축 해제**: 새 버전을 별도 디렉토리에 압축 해제
4. **전환**: 기존 런타임 중지 후 새 버전으로 전환
5. **정리**: 구 버전 정리

버전 정리

```go
func (m *Manager) cleanBundles(runtimeID common.Namespace) {
maxVersion, ok := func() (version.Version, bool) {
m.mu.Lock()
```

```
defer m.mu.Unlock()
maxVersion, ok := m.cleanupQueue[runtimeID]
if !ok {
return version.Version{}, false
}
delete(m.cleanupQueue, runtimeID)
return maxVersion, true
}()
if !ok {
return
}
// 최대 버전보다 낮은 버전들 정리
for _, manifest := range m.store.Manifests() {
if manifest.ID != runtimeID {
continue
}
ronl, ok := manifest.GetComponentByID(component.ID_RONL)
if !ok {
continue
}
if !ronl.Version.Less(maxVersion) {
continue
}
```

```
m.cleanBundle(manifest)

}

}
```

설정 및 사용 예제

번들 생성

번들 생성 도구

Oasis Core는 런타임 번들을 생성하기 위한 도구를 제공합니다.

```
# 번들 생성 도구 사용법

oasis-node  bundle  create \

--runtime-id <RUNTIME_ID> \

--runtime-version  1.0.0 \

--elf-executable runtime.elf \

--sgx-executable  runtime.sgxs \

--sgx-signature runtime.sig \

--output  runtime.orc
```

프로그래밍 방식 번들 생성

```
package main


import (
```

```go
"fmt"
"github.com/oasisprotocol/oasis-core/go/common"
"github.com/oasisprotocol/oasis-core/go/common/crypto/hash"
"github.com/oasisprotocol/oasis-core/go/common/version"
"github.com/oasisprotocol/oasis-core/go/runtime/bundle"
"github.com/oasisprotocol/oasis-core/go/runtime/bundle/component"
)

func  createRuntimeBundle(runtimeID common.Namespace) error {
// 새 번들 생성
bnd := &bundle.Bundle{
Manifest: &bundle.Manifest{
Name: "example-runtime",
ID: runtimeID,
Components: []*bundle.Component{
{
Kind: component.RONL, // Runtime On-Network Layer
Name: "main",
Version: version.Version{Major: 1, Minor: 0, Patch: 0},
ELF: &bundle.ELFMetadata{
Executable: "runtime.elf",
},
SGX: &bundle.SGXMetadata{
```

```
Executable: "runtime.sgxs",
Signature: "runtime.sig",
},
// 예상 엔클레이브 신원 (선택사항)
Identities: []bundle.Identity{
{
Enclave: sgx.EnclaveIdentity{
MrEnclave: expectedMrEnclave,
MrSigner: expectedMrSigner,
},
},
},
},
},
Digests: make(map[string]hash.Hash),
},
Data: make(map[string]bundle.Data),
}
// 런타임 파일들 추가
files := map[string]string{
"runtime.elf": "path/to/runtime.elf",
"runtime.sgxs": "path/to/runtime.sgxs",
"runtime.sig": "path/to/runtime.sig",
```

```go
}
for bundleName, filePath := range files {
if err := bnd.Add(bundleName, bundle.NewFileData(filePath)); err != nil {
return fmt.Errorf("파일 추가 실패 %s: %w", bundleName, err)
}
fmt.Printf("파일 추가됨: %s -> %s\n", filePath, bundleName)
}
// 번들 검증
if err := bnd.Validate(); err != nil {
return fmt.Errorf("번들 검증 실패: %w", err)
}
// 번들 저장
outputPath := "runtime.orc"
if err := bnd.Write(outputPath); err != nil {
return fmt.Errorf("번들 저장 실패: %w", err)
}
fmt.Printf("번들 생성 완료: %s\n", outputPath)
fmt.Printf("매니페스트 해시: %s\n", bnd.Manifest.Hash().Hex())
return nil
}

// 번들 정보 출력
func printBundleInfo(bundlePath string) error {
```

```go
bnd, err := bundle.Open(bundlePath)
if err != nil {
return fmt.Errorf("번들 열기 실패: %w", err)
}
defer bnd.Close()
fmt.Printf("번들 정보:\n")
fmt.Printf(" 이름: %s\n", bnd.Manifest.Name)
fmt.Printf(" ID: %s\n", bnd.Manifest.ID)
fmt.Printf(" 매니페스트 해시: %s\n", bnd.Manifest.Hash().Hex())
fmt.Printf(" 컴포넌트:\n")
for id, comp := range bnd.Manifest.GetAvailableComponents() {
fmt.Printf(" %s:\n", id)
fmt.Printf(" 종류: %s\n", comp.Kind)
fmt.Printf(" 버전: %s\n", comp.Version)
if comp.ELF != nil {
fmt.Printf(" ELF 실행파일: %s\n", comp.ELF.Executable)
}
if comp.SGX != nil {
fmt.Printf(" SGX 실행파일: %s\n", comp.SGX.Executable)
fmt.Printf(" SGX 서명파일: %s\n", comp.SGX.Signature)
}
}
return nil
```

```
}
```

번들 로딩

번들 매니저 사용

```
// 번들 매니저 생성
manager, err := NewManager(
dataDir,
[]common.Namespace{runtimeID},
store,
volumeManager,
)
if err != nil {
return fmt.Errorf("failed to create bundle manager: %w", err)
}

// 번들 매니저 시작
manager.Start()
defer manager.Stop()

// 번들 추가
err = manager.Add("runtime.orc", WithBundleLabels(map[string]string{
"version": "1.0.0",
```

```
"env": "production",
})) 
if err != nil {
return fmt.Errorf("failed to add bundle: %w", err)
}
```

런타임 로더 실행

SGX 런타임 로딩

```
# SGX 런타임 로더 실행
oasis-core-runtime-loader \
--type sgxs \
--signature /path/to/runtime.sig \
--host-socket /tmp/runtime-host.sock \
--allow-network \
/path/to/runtime.sgxs
```

ELF 런타임 로딩

```
# ELF 런타임 로더 실행 (sandbox 환경)
oasis-core-runtime-loader \
--type elf \
--host-socket /tmp/runtime-host.sock \
```

```
/path/to/runtime.elf
```

보안 고려사항

번들 무결성

1. 암호화 해시 검증

- 모든 번들 파일의 SHA-256 해시 검증
- 매니페스트에 저장된 다이제스트와 비교
- 변조 탐지 및 방지

2. 디지털 서명 검증

- SGX 엔클레이브의 SIGSTRUCT 검증
- MRENCLAVE와 MRSIGNER 확인
- 신뢰할 수 있는 서명자 검증

3. 번들 크기 제한

```
const (
maxDefaultBundleSizeBytes = 20 * 1024 * 1024  // 20 MB
maxMetadataSizeBytes = 2 * 1024  // 2 KB
)
```

네트워크 보안

1. HTTPS 전용 다운로드

- 모든 번들 다운로드는 HTTPS를 통해서만 수행
- TLS 인증서 검증 필수

2. 레지스트리 URL 검증

```
func validateAndNormalizeURL(rawURL string) (string, error) {
parsedURL, err := url.Parse(rawURL)
if err != nil {
return "", fmt.Errorf("invalid URL '%s': %w", rawURL, err)
}
// HTTPS만 허용
if parsedURL.Scheme != "https" {
return "", fmt.Errorf("only HTTPS URLs are allowed")
}
return parsedURL.String(), nil
}
```

실행 환경 격리

1. SGX 엔클레이브 격리

- 하드웨어 수준의 메모리 보호
- 호스트 OS로부터 완전 격리
- 원격 증명을 통한 무결성 검증

2. Sandbox 격리

- 프로세스 네임스페이스 분리
- 파일 시스템 접근 제한
- 네트워크 접근 제어

모니터링 및 디버깅

로그 및 메트릭

번들 관리 로그

```
m.logger.Info("downloading bundle",
"url", url,
"manifest_hash", manifestHash.Hex(),
)
```

```
m.logger.Info("bundle exploded",
"dir", dir,
"manifest_hash", manifest.Hash().Hex(),
)
```

```
m.logger.Error("failed to download bundle",
"err", err,
"runtime_id", runtimeID,
"manifest_hash", hash.Hex(),
```

```
)
```

런타임 로더 로그

```
log::info!("Starting SGX runtime loader");

log::info!("Runtime file: {}", filename);

log::info!("Signature file: {:?}", signature_filename);

log::info!("Host socket: {}", host_socket);

log::info!("Network allowed: {}", allow_network);
```

문제 해결

일반적인 문제

1. **번들 다운로드 실패**

- 네트워크 연결 확인
- 레지스트리 URL 검증
- 인증서 문제 확인

2. **번들 검증 실패**

- 파일 무결성 확인
- 서명 검증 상태 확인
- 매니페스트 형식 검증

3. **런타임 로딩 실패**

- SGX 드라이버 설치 확인
- 파일 권한 확인

- 의존성 라이브러리 확인

디버깅 도구

```
# 번들 내용 확인
unzip -l runtime.orc

# 매니페스트 내용 확인
unzip -p runtime.orc META-INF/MANIFEST.MF | jq .

# SGX 디바이스 확인
ls -la /dev/sgx* /dev/isgx

# 런타임 로더 디버그 실행
RUST_LOG=debug oasis-core-runtime-loader --type sgxs ...
```

성능 모니터링

```
# 번들 다운로드 성능 측정
time oasis-node bundle download --runtime-id <ID> --manifest-hash <HASH>

# 런타임 로딩 시간 측정
time oasis-core-runtime-loader --type sgxs --signature runtime.sig \
--host-socket /tmp/host.sock runtime.sgxs
```

```
# 메모리 사용량 모니터링
ps aux | grep oasis-core-runtime-loader
cat /proc/$(pidof oasis-core-runtime-loader)/status | grep VmRSS
```

로그 분석

```
# 번들 매니저 로그 필터링
journalctl -u oasis-node | grep "runtime/bundle"

# 런타임 로더 로그 분석
RUST_LOG=info oasis-core-runtime-loader 2>&1 | tee runtime-loader.log

# 오류 패턴 검색
grep -E "(ERROR|WARN|failed|error)" runtime-loader.log
```

고급 주제

커스텀 번들 검증

사용자 정의 검증 로직

```
// 커스텀 번들 검증 함수
func customBundleValidator(bnd *bundle.Bundle) error {
// 1. 런타임 크기 제한 확인
```

```go
for _, comp := range bnd.Manifest.GetAvailableComponents() {
if comp.ELF != nil {
data, ok := bnd.Data[comp.ELF.Executable]
if !ok {
continue
}
size, err := getDataSize(data)
if err != nil {
return fmt.Errorf("크기 확인 실패: %w", err)
}
if size > maxRuntimeSize {
return fmt.Errorf("런타임 크기 초과: %d bytes", size)
}
}
}
// 2. 허용된 서명자 확인
for _, comp := range bnd.Manifest.GetAvailableComponents() {
if comp.SGX != nil {
mrSigner, err := bnd.MrSigner(comp.ID())
if err != nil {
return fmt.Errorf("MRSIGNER 확인 실패: %w", err)
}
if !isAllowedSigner(*mrSigner) {
```

```go
return fmt.Errorf("허용되지 않은 서명자: %s", mrSigner)
}
}
}
// 3. 런타임 버전 호환성 확인
for _, comp := range bnd.Manifest.GetAvailableComponents() {
if !isCompatibleVersion(comp.Version) {
return fmt.Errorf("호환되지 않는 버전: %s", comp.Version)
}
}
return nil
}

// 번들 매니저에 커스텀 검증 적용
err = manager.Add("runtime.orc",
WithBundleValidator(customBundleValidator),
WithBundleLabels(map[string]string{
"environment": "production",
"validated": "true",
}),
)
```

번들 캐싱 최적화

지능형 캐싱 전략

```go
type BundleCache struct {
cache  map[hash.Hash]*CacheEntry
lru *list.List
maxSize  int64
currSize  int64
mu sync.RWMutex
}

type CacheEntry struct {
bundle *bundle.Bundle
size  int64
lastUsed time.Time
element *list.Element
}

func (c *BundleCache) Get(manifestHash hash.Hash) (*bundle.Bundle, bool) {
c.mu.Lock()
defer  c.mu.Unlock()
entry, exists := c.cache[manifestHash]
if !exists {
return  nil, false
```

```go
}
// LRU 업데이트
c.lru.MoveToFront(entry.element)
entry.lastUsed = time.Now()
return  entry.bundle, true
}


func (c *BundleCache) Put(manifestHash hash.Hash, bnd *bundle.Bundle) {
c.mu.Lock()
defer  c.mu.Unlock()
size := c.calculateBundleSize(bnd)
// 캐시 공간 확보
for  c.currSize+size > c.maxSize && c.lru.Len() > 0 {
c.evictLRU()
}
// 새 엔트리 추가
element := c.lru.PushFront(manifestHash)
entry := &CacheEntry{
bundle: bnd,
size: size,
lastUsed: time.Now(),
element: element,
}
```

```
c.cache[manifestHash] = entry
c.currSize += size
}
```

분산 번들 레지스트리

다중 레지스트리 관리

```
type RegistryManager struct {
registries []Registry
strategy LoadBalanceStrategy
health  map[string]RegistryHealth
}

type Registry struct {
URL  string
Priority  int
Timeout time.Duration
Auth AuthConfig
}

type RegistryHealth struct {
Available  bool
LastCheck time.Time
```

```go
ResponseTime time.Duration
ErrorCount  int
}


func (rm *RegistryManager) DownloadBundle(manifestHash hash.Hash) (*bundle.Bundl
// 건강한 레지스트리 선택
registry := rm.selectHealthyRegistry()
if  registry == nil {
return  nil, fmt.Errorf("사용 가능한 레지스트리가 없음")
}
// 다운로드 시도
bnd, err := rm.downloadFromRegistry(registry, manifestHash)
if  err != nil {
// 실패시 다른 레지스트리로 폴백
return  rm.downloadWithFallback(manifestHash, registry)
}
return  bnd, nil
}
```

런타임 로더는 Oasis Core의 런타임 실행을 위한 핵심 인프라로, 안전하고 효율적인 런타임 배포와 실행을 보장합니다. 번들 관리, 검증, 로딩의 전 과정에서 보안과 성능을 모두 고려한 설계를 통해 신뢰할 수 있는 런타임 환경을 제공합니다.